

CLDRoute: Conditional Latent Diffusion for Routability Map Generation in Physical Design

Kiran Thorat^{1*} Nicole Meng^{2*} Caiwen Ding³ Yingjie Lao² Zhijie Jerry Shi¹

¹University of Connecticut

²Tufts University

³University of Minnesota Twin Cities

{kiran_gautam.thorat,zshi}@uconn.edu {ziyi.meng,Yingjie.lao}@tufts.edu dingc@umn.edu

Abstract

Accurate routability estimation during physical design is important for reducing costly post-routing iterations. Prior learning-based methods treat this task as deterministic prediction, mapping placement-stage features to a single congestion or DRC outcome. We instead formulate routability estimation as a *conditional generation* problem, where both routing congestion and DRC violations are modeled as spatially structured *routability fields*. Our framework, *Conditional Latent Diffusion for Routability estimation (CLDRoute)*, uses physics-aware conditioning and task-specific latent modeling to handle the different characteristics of congestion and DRC maps. This allows our method to support sample-based inference, producing both a mean prediction and a spatial uncertainty estimate for the same input design. On CircuitNet 2.0 (N28), our method achieves, for DRC violation generation, an SSIM of 0.9678, an MAE of 0.0028, and a TopK@1% of 0.3494; for congestion generation, it achieves an SSIM of 0.9031, an MAE of 0.0286, and an NZ-Pearson of 0.3692. Overall, our framework provides a more practical view of routability at placement by generating both the expected outcome and its uncertainty.

Keywords

Generative modeling, physical design, diffusion models

1 Introduction

Routability estimation at the placement stage has received increasing attention, as routing congestion and design-rule-check (DRC) violations identified after routing often trigger costly iterations across placement, routing, and verification. As design rules and interconnect complexity continue to increase, late-stage failures remain a major source of design closure cost. This has motivated substantial research on predicting routing outcomes prior to detailed routing. [2, 6].

Existing learning-based methods primarily formulate routability as a prediction task. Given a design representation, the model predicts a routing congestion map, a DRC-related target, or a post-routing property [20, 24]. However, the dominant formulation remains to regress a single output map or scalar target from a placed design.

The main gap lies in the formulation itself. At the placement stage, the final routing solution is not yet available. A placed design can admit multiple valid downstream routing outcomes, as congestion resolution, detours, layer assignment, and detailed routability constraints are only resolved during routing. Routability is therefore not a single fixed target at placement, but rather a conditional

distribution over plausible routing outcomes. A deterministic predictor compresses this distribution into a single estimate and cannot capture spatial uncertainty [9, 23, 24]. In this sense, routability is better formulated as *conditional generation* rather than deterministic regression. In this work, we model both the routing congestion map and the DRC violation map as generated *routability fields*.

A second gap lies in the conditioning representation. Many existing methods rely on a limited set of placement-derived channels, such as macro-region and RUDY-style features [2, 6, 24]. In this work, we introduce *physics-aware conditioning channels* to guide the model using routing-relevant spatial signals that are more directly tied to routing demand and available resources. The goal is not only to predict a single map, but to generate a distribution of plausible routing outcomes under physically meaningful conditioning.

Based on these observations, we propose a conditional generative framework for routability estimation. The framework models both the routing congestion map and the DRC violation map within a unified pipeline, while allowing the two targets to retain their distinct output characteristics. Instead of returning a single prediction, it supports sample-based inference, producing both a mean prediction and a spatial uncertainty map. The goal is to move routability estimation from single-output prediction to distributional modeling that better reflects the nature of the physical-design problem. The main contributions are summarized as follows:

- **Distributional formulation of routability:** Routability is designed as conditional generation rather than deterministic prediction in our method, modeling congestion and DRC as stochastic *routability fields*. This formulation captures the inherent ambiguity of placement-stage signals and exposes spatial uncertainty that is inaccessible to regression-based methods.
- **Physics-aware spatial conditioning:** CLDRoute’s conditioning is structured through routing-relevant signals tied to demand and resource constraints, moving beyond generic placement features. This provides direct control over the generation process and grounds predictions in physically meaningful cues.
- **Task-aligned latent modeling:** Congestion and DRC targets are represented in separate latent spaces matched to their distinct statistical properties, allowing high-resolution modeling without sacrificing efficiency or fidelity.
- **Controllable generative pipeline with uncertainty:** A multi-scale ControlNet-style latent diffusion framework produces both accurate predictions and calibrated spatial uncertainty maps, offering actionable signals for identifying high-risk regions during design.

*Equal Contribution.

[†]Code is available at github.com/kiranthorat3/CLDRoute.

Table 1: Comparison of routability methods.

Method	Physics-aware controls	Both tasks	Conditional generation	Spatial uncertainty
LayNet [24]	✗	✓	✗	✗
GPDL [9]	✗	✓	✗	✗
RouteNet [20]	✗	✗	✗	✗
CLDRoute (ours)	✓	✓	✓	✓

As shown in Table 1, existing methods primarily map placement-stage features to a single deterministic routability output. In contrast, CLDRoute models routability as a conditional distribution over plausible congestion and DRC maps, and produces spatial uncertainty through sample-based inference. To the best of our knowledge, CLDRoute is the first framework to formulate placement-stage routability estimation in this way for both tasks. This formulation better reflects the ambiguity of routing outcomes at placement stage and motivates the physics-aware, task-specific design of our framework

2 Background

The physical design stage in EDA involves large-scale NP-hard optimization, where placement and routing jointly determine design quality [13]. Routing resolves wire assignments, layer usage, and design-rule constraints, and is often the most time-consuming stage. Predicting routing outcomes at placement has therefore become important for reducing costly design iterations.

2.1 Routability Estimation

Routability estimation aims to predict post-routing properties, such as routing congestion and design-rule-check (DRC) violations, from placement-stage representations. These predictions guide placement optimization by identifying regions likely to cause routing failures. A common representation is the routing congestion map, which captures routing demand relative to available resources across the layout grid, while DRC-related targets identify potential violations under detailed routing constraints.

Most existing approaches treat routability estimation as a supervised prediction task, where a model maps placement-derived features to a single congestion map or scalar metric [19]. However, this formulation assumes a one-to-one mapping between placement and routing outcomes, which does not reflect the variability introduced by routing decisions such as detours, layer assignment, and local constraint resolution.

2.2 Learning-based Routability Prediction

DNN-based methods. Deep learning approaches typically convert layouts into image representations and apply fully convolutional networks (FCN) to predict routability [9, 20]. For example, GPDL [20] adopts an encoder-decoder architecture to map grid features to congestion maps, while RouteNet [19] uses CNN-based modeling for routing-related prediction. Generative models in computer vision [12] such as GANs [8, 22] and conditional graph attention networks [22] have also been explored for congestion forecasting. However, these methods are often limited by resolution and by the use of incomplete domain-specific features.

GNN-based methods. Graph neural network approaches construct graphs from netlists and layout features, encoding both connectivity and spatial information [1, 3, 21]. These methods leverage both topological and geometric signals to predict congestion and related metrics. While they provide a more structured representation, their effectiveness depends on graph construction quality and they may not explicitly model routing resource constraints.

Across both image-based and graph-based methods, they focus mainly on deterministic prediction of a single output. This formulation does not capture the inherent uncertainty of routing outcomes at the placement stage, where multiple valid routing solutions may exist. In addition, conditioning is often treated as an unstructured collection of input channels, without explicitly incorporating routing-relevant physical signals. These limitations motivate a formulation that models routability as a distribution over plausible outcomes, conditioned on physically meaningful design features.

3 Framework

As illustrated in Fig. 1, the framework proceeds in three stages. Fig. 1(a) shows how placement-stage design files are converted into spatial routing controls. Fig. 1(b) shows the two task-specific VAEs used to encode DRC and congestion maps into separate latent spaces. Fig. 1(c) shows the conditional latent diffusion model, where the noisy latent is denoised under multi-scale routing controls and then decoded to produce the final routability map.

3.1 Physics-Aware Routing Controls

The quality of conditional generation depends directly on the quality of its routing controls. Prior routability models usually reuse feature sets from benchmark pipelines, where the channels are selected mainly for availability and for regression-based prediction [2, 6, 9, 20]. In this work, we instead design the routing controls from physical-design reasoning. The objective is not simply to add more channels, but to expose the model to spatial signals that are directly related to routability.

Congestion and DRC depend on different physical cues, so they should not use the same routing controls. Congestion is governed mainly by the spatial balance between routing demand and available routing resources across the layout. DRC depends more strongly on local access pressure, routing conflicts, and geometric context near obstacles and dense placement regions. This motivates task-specific control design: congestion uses controls that emphasize layout-wide demand-resource balance, while DRC uses a broader set of local conflict cues.

We organize the routing controls into three physical groups,

$$\mathcal{F}_{\text{task}} = \mathcal{F}_{\text{demand}} \cup \mathcal{F}_{\text{supply}} \cup \mathcal{F}_{\text{geometry}}, \quad (1)$$

where $\mathcal{F}_{\text{demand}}$ contains channels that describe where routing pressure originates, $\mathcal{F}_{\text{supply}}$ contains channels that describe the available routing resources and their usage state, and $\mathcal{F}_{\text{geometry}}$ contains channels that describe the placement structure that shapes routing difficulty. In our implementation, the demand group includes RUDY-based and pin-related demand maps [18]; the supply group includes GR and eGR utilization or overflow-related signals from the routing flow [10, 14]; and the geometry group includes macro

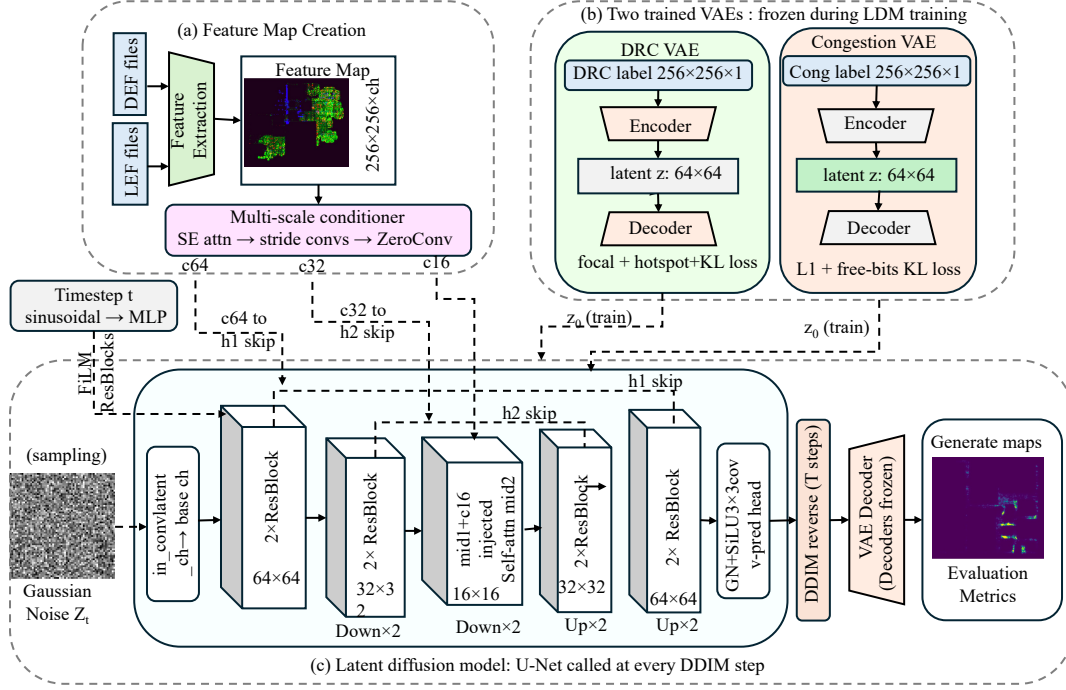


Figure 1: Overview of the proposed conditional latent diffusion framework for routability prediction. (a) Routing features extracted from DEF/LEF files are passed through a multi-scale conditioner producing spatial conditioning signals c_{64} , c_{32} , and c_{16} . (b) Two task-specific VAEs compress DRC and congestion labels into latent representations and are frozen during LDM training. (c) A conditioned U-Net denoiser generates latent codes via DDIM sampling with classifier-free guidance; the frozen VAE decoder produces the final routability map.

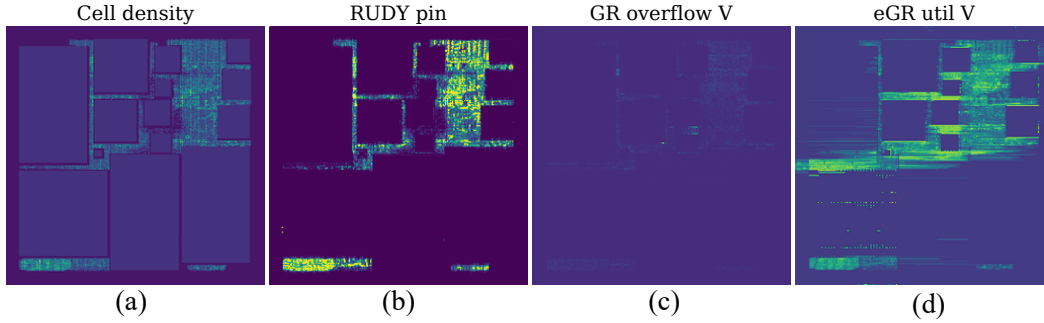


Figure 2: Representative DRC routing controls for one test design: (a) cell density, (b) RUDY_pin, (c) GR_overflow_V, and (d) eGR_util_V. Together, these controls provide geometry, demand, and supply information used by the model.

obstacles, cell density, and macro-boundary context. This grouping keeps the conditioning design simple, consistent, and physically grounded.

Table 2 lists the final task-specific control sets. The expanded control banks contain 11 channels for congestion and 16 channels for DRC. After removing one inactive channel per task before training, the model uses 10 effective congestion controls and 15 effective DRC controls. For congestion, the control set combines demand and supply signals so that the model can observe both

where wires tend to accumulate and how much routing capacity remains available. For DRC, the control set retains demand, utilization, overflow, and geometry-related signals, since DRC violations depend on more localized interactions among these factors. In both tasks, the original CircuitNet controls are retained, and the added controls extend them with information that is more directly tied to routing resources and geometric constraints.

To support this design, we compute the Pearson correlation between each routing-control channel f_c and the target label y over

Table 2: Task-specific routing controls used in the experiments. The reported correlation is the global Pearson correlation $r_c = \text{corr}(f_c, y)$ on the N28 expanded dataset. Channels are grouped by physical role.

Task	Channel	Group	Corr. r_c
Cong	macro_region	Geometry	-0.2768
	RUDY	Demand	0.3855
	RUDY_pin	Demand	0.4358
	RUDY_long	Demand	0.3754
	RUDY_short	Demand	0.2577
	cell_density	Geometry	0.3885
	GR_util_H	Supply	0.4831
	GR_util_V	Supply	0.4007
	eGR_overflow_H	Supply	0.0404
	eGR_overflow_V	Supply	0.0900
DRC	macro_region	Geometry	-0.1016
	cell_density	Geometry	0.1396
	RUDY_long	Demand	0.1748
	RUDY_short	Demand	0.0949
	RUDY_pin_long	Demand	0.1485
	eGR_overflow_H	Supply	0.0407
	eGR_overflow_V	Supply	0.0640
	GR_overflow_H	Supply	0.2681
	GR_overflow_V	Supply	0.2865
	GR_util_H	Supply	0.1999
	GR_util_V	Supply	0.1524
	RUDY	Demand	0.1774
	RUDY_pin	Demand	0.1503
	eGR_util_H	Supply	0.2112
	eGR_util_V	Supply	0.2236

all pixels,

$$r_c = \text{corr}(f_c, y). \quad (2)$$

For congestion, the strongest global correlations appear in both supply and demand channels, including GR_util_H ($r = 0.4831$), RUDY_pin ($r = 0.4358$), GR_util_V ($r = 0.4007$), and cell_density ($r = 0.3885$). This supports using routing controls that expose both routing pressure and remaining routing resources. For DRC, no single channel dominates. Instead, the signal is distributed across several moderate-correlation channels, including GR_overflow_V ($r = 0.2865$), GR_overflow_H ($r = 0.2681$), eGR_util_V ($r = 0.2236$), and GR_util_H ($r = 0.1999$). This supports using a broader task-specific control set for DRC. Additional dataset analysis is used in the next section to motivate the latent design for each task.

All routing controls are rasterized on the same spatial grid and normalized to the same range before they are passed to the model. Figure 2 shows representative examples from these control groups together with the target map. The main point of this subsection is that the routing controls are selected from physically meaningful signal groups and tailored to the different structures of congestion and DRC.

3.2 Task-Specific Latent Encoding for Routability Fields

Raw pixel space is not an ideal domain for routability-field generation. A natural alternative is to first encode the target map into a compact latent representation with a variational autoencoder (VAE) [7], and then perform diffusion in that latent space [15]. In our setting, however, a single generic latent design is not well matched to both targets.

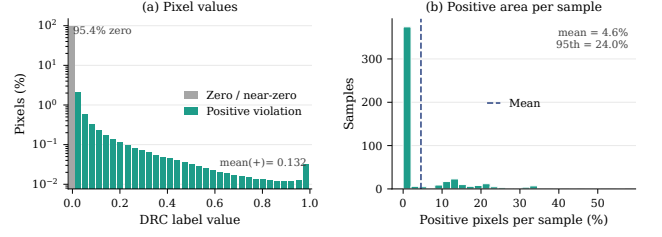


Figure 3: Distribution of DRC labels in the training set. The target is highly sparse: most pixels are zero or near-zero, and the positive violation area varies substantially across designs.

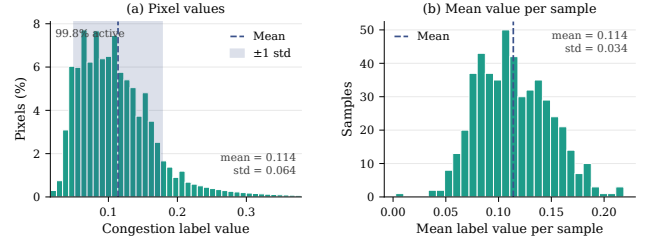


Figure 4: Distribution of congestion labels in the training set. The target is dense and concentrated around a relatively narrow value range, with moderate variation across designs.

The main reason is that DRC and congestion have very different label structures. As shown in Fig. 3(a), the DRC target is highly sparse: 95.4% of pixels are zero, the mean positive value is 0.132, and the mean positive area per sample is only 4.6% (Fig. 3(b)). In contrast, Fig. 4(a) shows that the congestion target is dense: 99.8% of pixels are active, with values concentrated around a mean of 0.114 and standard deviation of 0.064 (Fig. 4(b)). These different target statistics lead to different latent-design requirements. For DRC, the main challenge is to preserve sparse violation structure and rare high-value regions. For congestion, the main challenge is to represent a dense spatial field faithfully while maintaining a stable latent space for generation. This motivates task-specific latent encoding for the two routability fields.

We therefore use *task-specific* latent compression before diffusion. In both tasks, the encoder maps a $1 \times 256 \times 256$ routing map to a latent posterior at 64×64 resolution, and the decoder maps the latent back to a $1 \times 256 \times 256$ output map. The latent resolution is chosen from the data. A $4\times$ spatial reduction from 256×256 to 64×64 preserves essentially all task-relevant signal for both targets, whereas a more aggressive $8\times$ reduction to 32×32 begins to discard sparse DRC structure. The VAE is therefore not only a computational tool; it is the interface that converts each routing target into a generation domain better matched to its statistics.

Both encoders use the same high-level objective

$$\mathcal{L}_{\text{VAE}}^{(\text{task})} = \mathcal{L}_{\text{recon}}^{(\text{task})} + \beta \mathcal{L}_{\text{KL}}, \quad (3)$$

where the reconstruction term depends on the task and the KL term keeps the latent compatible with the Gaussian prior used by the

diffusion model. We use per-channel free bits [7]:

$$\mathcal{L}_{\text{KL}} = \frac{1}{C_z} \sum_{c=1}^{C_z} \max(\lambda, \text{KL}_c), \quad \lambda = 0.5 \text{ nats}, \quad (4)$$

where C_z is the number of latent channels and KL_c is the KL divergence of channel c . This prevents individual channels from collapsing too early and keeps the latent usable for diffusion.

3.2.1 Encoding Sparse DRC Violation Maps. The DRC violation map requires a violation-focused latent design. The difficulty is not only that nonzero violations are rare, but also that most of the map is zero background. A uniform reconstruction loss in raw pixel space would therefore spend most of its capacity on background pixels instead of the spatial violation pattern.

We address this in three steps. First, we encode the raw DRC violation map $x_{\text{DRC}} \in [0, 1]^{256 \times 256}$ in log space:

$$\tilde{x}_{\text{DRC}} = \log(1 + 10 x_{\text{DRC}}). \quad (5)$$

This expands the low-value nonzero region before compression and makes small violations easier to separate from the background.

Second, the decoder reconstructs the output map $\hat{x}_{\text{DRC}} \in [0, 1]^{256 \times 256}$ with a violation-weighted loss:

$$\mathcal{L}_{\text{focal}}^{\text{DRC}} = \frac{1}{HW} \sum_{i=1}^{HW} \left(1 + \gamma \sqrt{x_{\text{DRC},i}}\right) (\hat{x}_{\text{DRC},i} - x_{\text{DRC},i})^2, \quad \gamma = 20. \quad (6)$$

This increases the training signal on nonzero violation pixels and reduces the dominance of the zero background.

Third, we add a hotspot term on the top 1% highest-valued DRC pixels in each sample:

$$\mathcal{L}_{\text{hot}}^{\text{DRC}} = \text{MSE}\left(\hat{x}_{\text{DRC}}^{\text{top 1\%}}, x_{\text{DRC}}^{\text{top 1\%}}\right), \quad (7)$$

and use the full DRC reconstruction objective

$$\mathcal{L}_{\text{recon}}^{\text{DRC}} = \mathcal{L}_{\text{focal}}^{\text{DRC}} + 0.5 \mathcal{L}_{\text{hot}}^{\text{DRC}}. \quad (8)$$

The DRC encoder compresses the $1 \times 256 \times 256$ input map to a $12 \times 64 \times 64$ latent and uses $\beta = 0.05$ with a 15-epoch warmup. The purpose of this design is to preserve rare violation structure while still producing a latent compatible with diffusion. Table 4 shows that all 12 latent channels remain active, with posterior standard deviations in the narrow range [0.630, 0.642], indicating that the DRC latent stays well distributed instead of collapsing to a few channels.

3.2.2 Encoding Dense Congestion Maps. The congestion map requires a different latent design. As shown in Fig. 4, the congestion label is dense and smooth, not sparse. The main failure mode is therefore not background dominance, but posterior collapse: if the KL pressure is too strong, the encoder can represent the dense congestion field with only a few dominant channels and let the remaining channels match the prior [5, 11].

For this reason, we keep the reconstruction term simple. Let $x_{\text{cong}} \in [0, 1]^{256 \times 256}$ be the raw congestion map and $\hat{x}_{\text{cong}} \in [0, 1]^{256 \times 256}$ its reconstruction. We use

$$\mathcal{L}_{\text{recon}}^{\text{cong}} = \frac{1}{HW} \sum_{i=1}^{HW} |\hat{x}_{\text{cong},i} - x_{\text{cong},i}|. \quad (9)$$

No log transform, focal weighting, or hotspot term is used for congestion.

Table 3: Effect of KL design on congestion latent quality. Active channels have $\sigma_c > 0.10$. The final design keeps all 8 channels active and produces a latent suitable for diffusion.

Configuration	β	Free-bits (λ)	Active channels	σ_c range
Standard VAE	0.020	None	3/8	[0.008, 0.480]
Ours	0.005	0.5 nats	8/8	[0.784, 0.801]

Table 4: Latent channel statistics before normalization for both task-specific encoders. Near-zero channel means and narrow standard-deviation ranges indicate that both latents are well behaved before handoff to the conditional diffusion model.

Encoder	Task	C_z	σ_c range	$\max \mu_c $	Global $\bar{\sigma}$
DRC encoder	DRC	12	[0.630, 0.642]	0.008	0.637
Congestion encoder	Congestion	8	[0.784, 0.801]	0.017	0.792

The main design effort is placed on the latent regularization. The congestion encoder compresses the $1 \times 256 \times 256$ input map to an $8 \times 64 \times 64$ latent. We again use the free-bits KL term in Eq. (4), but with a smaller KL weight, $\beta = 0.005$, and a longer 40-epoch warmup. This reduces the pressure to compress the dense field into only a few latent dimensions.

Table 3 shows why this design is necessary. With a stronger KL setting, only 3 of 8 channels remain active. With the final design, all 8 channels remain active, with posterior standard deviations in the narrow range [0.784, 0.801]. For congestion, the VAE therefore solves a different problem than for DRC: not rare-event preservation, but stable dense-field compression without latent collapse.

Table 4 reports the latent statistics before normalization. In both tasks, the channel means are already close to zero and the channel standard deviations lie in a narrow range. After normalization, the conditional diffusion model receives a standardized latent target for both DRC and congestion. The final latent targets for diffusion are therefore $12 \times 64 \times 64$ for DRC violation maps and $8 \times 64 \times 64$ for congestion maps. These latents preserve the task-relevant spatial signal while moving generation away from the raw pixel domain.

3.3 Conditional Latent Diffusion with Multi-Scale Routing Controls

After learning a task-specific latent space for each target map, we modeled routability-map generation with a conditional latent diffusion model. Latent diffusion is a standard framework for generative modeling in compressed latent spaces [4, 15, 16]. Our focus here was the conditioning design for routability generation. Instead of treating placement-stage routing features as a single appended input, we used them as explicit *routing controls* that guided generation at multiple spatial scales.

Let $x \in \mathbb{R}^{1 \times 256 \times 256}$ denote the target routability map and let $f \in \mathbb{R}^{C_{\text{feat}} \times 256 \times 256}$ denote the routing feature tensor. The task-specific VAE encoder mapped x to a compact latent code $z_0 \in \mathbb{R}^{C_z \times 64 \times 64}$, and the diffusion model learned the conditional distribution $p(z_0 | f)$. The denoising UNet took only the noisy latent as input. The routing features were processed by a separate control branch and injected

into the UNet internally. This design made the conditioning role explicit: the latent represented *what* should be generated, while the routing controls determined *how* that generation should follow the physical structure of the design.

A key observation was that routing pressure is naturally multi-scale. Global demand affects the coarse layout of congestion and violation regions, intermediate demand shapes regional structure, and local overflow-related signals influence the precise placement of hotspots. For this reason, we did not use a single conditioning tensor. Instead, as shown in Fig. 1(c), we transformed the routing features into three control tensors, c_{64} , c_{32} , and c_{16} , and injected them into the corresponding stages of the latent UNet.

Multi-scale routing controls. A lightweight feature encoder transforms the routing feature map into three spatial resolutions,

$$(f_{64}, f_{32}, f_{16}) = C(f), \quad (10)$$

where the subscripts denote spatial size. Each feature map is then projected to the channel dimension required by the corresponding UNet stage using a zero-initialized 1×1 convolution,

$$c_s = W_s * f_s, \quad s \in \{64, 32, 16\}, \quad (11)$$

with W_s initialized to zero. The resulting routing controls are injected at matched resolutions: c_{64} is added to the high-resolution skip connection, c_{32} is added to the intermediate skip connection, and c_{16} is injected at the bottleneck before the middle attention block. This lets the model use coarse routing context to shape global structure and finer routing context to refine local map details.

The zero-initialized projections make the control branch stable to train. At initialization, all routing controls are exactly zero, so the network first behaves as a latent denoiser without feature guidance. As training proceeds, the control branch gradually learns how to modulate generation using routing signals. We also use classifier-free guidance by dropping all three control tensors together with probability p_{cfg} . Each training sample is therefore seen either with full routing control or with no routing control, which keeps the conditional and unconditional branches consistent across scales.

Given the normalized latent target $\hat{z}_0$, a timestep t , and Gaussian noise ϵ , we form the noisy latent $\hat{z}_t$ using the standard forward diffusion process. The denoising network v_θ then predicts the velocity target in latent space conditioned on the routing controls:

$$\mathcal{L}_{\text{ldm}} = \mathbb{E}_{t, \hat{z}_0, \epsilon} [\|v_\theta(\hat{z}_t, t; c_{64}, c_{32}, c_{16}) - v\|^2], \quad (12)$$

where

$$v = \sqrt{\alpha_t} \epsilon - \sqrt{1 - \alpha_t} \hat{z}_0. \quad (13)$$

This objective follows the standard latent diffusion formulation [4, 15, 16]. The task-specific component in our setting is the use of multi-scale routing controls, which guide denoising toward routability maps that remain consistent with placement-stage signals. Algorithm 1 summarizes the training and inference procedure of the task-specific routing-controlled latent diffusion model.

Sample-based generation and uncertainty. At inference time, we generate multiple latent samples for the same routing-control tensor f and decode them back to label space with the task-specific VAE decoder. The final routability estimate is the sample mean

$$\bar{x} = \frac{1}{N} \sum_{k=1}^N \text{Dec}(z^{(k)}), \quad (14)$$

Algorithm 1 Task-specific routing-controlled latent diffusion

Input: routing controls f , target routability map x , task-specific VAE encoder Enc , task-specific VAE decoder Dec , multi-scale conditioner C , denoiser v_θ

- 1: $z_0 \leftarrow \text{Enc}(x)$
 - 2: sample timestep t and Gaussian noise ϵ
 - 3: construct noisy latent z_t from z_0 , t , and ϵ
 - 4: $(f_{64}, f_{32}, f_{16}) \leftarrow C(f)$
 - 5: $(c_{64}, c_{32}, c_{16}) \leftarrow (W_{64} * f_{64}, W_{32} * f_{32}, W_{16} * f_{16})$
 - 6: apply joint classifier-free guidance dropout to (c_{64}, c_{32}, c_{16})
 - 7: $\hat{v} \leftarrow v_\theta(z_t, t; c_{64}, c_{32}, c_{16})$
 - 8: update model parameters using $\|\hat{v} - v\|^2$
 - 9: **Inference:** for a fixed routing-control tensor f , generate N latent samples $\{z^{(k)}\}_{k=1}^N$
 - 10: decode each sample $x^{(k)} \leftarrow \text{Dec}(z^{(k)})$
 - 11: return mean map $\bar{x} = \frac{1}{N} \sum_{k=1}^N x^{(k)}$ and variance map $\sigma^2 = \text{Var}_k[x^{(k)}]$
-

Table 5: Design-wise dataset splits used in the experiments.

Node	Train	Val	Test
N28	7,872	1,248	1,122
N14	10,368	169	250

and the accompanying uncertainty map is the per-pixel sample variance. This gives two outputs for the same placed design: a mean routability field and a spatial map showing where the generated outcome varies more across samples. From the application point of view, the model therefore provides both a likely routability map and a direct indication of where the placement-stage signals support multiple plausible outcomes.

4 Evaluation

Experimental Setup. All experiments were performed on a Linux system with an AMD EPYC 7763 64-core processor and four NVIDIA RTX A6000 GPUs (48 GB each). The task specific VAEs train with AdamW, learning rate 10^{-4} , cosine decay with 500-step linear warmup, EMA decay 0.999. The latent diffusion model with multi-scale ControlNet conditioning trains with the same optimizer and EMA settings $T = 1000$ diffusion steps, cosine noise schedule [4]. At inference, DDIM [17] with $N = 8$ independent draws per design, averaged in label space after VAE decoding.

Dataset. We evaluate on the CircuitNet 2.0 dataset [2, 6] at two technology nodes, N28 and N14, corresponding to 28 nm and 14 nm designs. We use a design-wise split for both congestion and DRC, so that training, validation, and test sets contain disjoint designs. The split sizes are summarized in Table 5.

4.1 Evaluation Metrics

We reported two groups of metrics. The first group consisted of standard regression metrics used for cross-method comparison. The second group consisted of task-specific metrics designed to better reflect how the generated routability fields would be used in physical design.

Standard metrics. We report MAE, NRMS, SSIM, and Pearson correlation. MAE and NRMS measured pixel-wise reconstruction

error, SSIM measured structural similarity between the generated and target maps, and Pearson correlation measured linear agreement over the full field. These metrics provided a common basis for comparison with prior work, but they did not fully capture sign-off-oriented behavior, especially when the target map was sparse.

Task-specific metrics. We additionally reported metrics that were aligned with the physical meaning of each task. For DRC, the main question was whether the model identified the correct violation regions and estimated their severity well. For congestion, the main question was whether the generated field preserved the spatial distribution of routing demand and remained well aligned in active regions. For DRC, we reported *TopK@1%* and *TopK@0.5%*, which measured the overlap between the highest-valued predicted locations and the highest-valued ground-truth locations. These metrics evaluated hotspot localization directly. We also reported *F1@0.1*, which summarized precision and recall at a fixed severity threshold, *Hotspot-MAE*, which measured error restricted to ground-truth hotspot regions, and *NZ-Pearson*, which computed correlation only on nonzero ground-truth pixels to reduce the effect of the dominant background. For congestion, we reported *NZ-Pearson*, *Spatial Bias*, and *Uncertainty-error correlation*. *NZ-Pearson* measured correlation in active regions of the congestion field. *Spatial Bias* measured the signed difference between the predicted and target global congestion levels, where values closer to zero indicated better overall calibration of the generated field. *Uncertainty-error correlation*, reported for generative models, measured the Pearson correlation between per-pixel predictive variance across multiple samples and the corresponding absolute error. Higher values indicated that regions with larger predictive uncertainty also tended to have larger generation error, making the uncertainty map more informative in practice. Overall, the task-specific metrics complemented the standard metrics by evaluating aspects of routability quality that were more directly related to downstream physical-design use.

4.2 Variational Autoencoder Evaluation

We evaluate the task-specific VAEs on held-out test data to verify that each model preserves the structure of its target label space before diffusion. Table 6 summarizes the reconstruction results on N28 and N14. On N28, both task-specific VAEs show high reconstruction fidelity. For DRC, the VAE reaches an MAE of 0.00095, an SSIM of 0.9934, and a correlation of 0.9870. For congestion, it reaches an MAE of 0.00132, an SSIM of 0.9932, and a correlation of 0.9608. These results indicate that the first-stage latent representations retain the main spatial structure of both routability fields on N28. On N14, the congestion VAE also preserves the dense spatial structure well, with an MAE of 0.00390, an SSIM of 0.9676, and a Pearson correlation of 0.9305. For DRC, the target maps remain much sparser, so the reported correlation is *NZ-Pearson*. In this setting, the DRC VAE reaches an MAE of 0.00648, an SSIM of 0.7009, and an *NZ-Pearson* of 0.2782. Together, these results show that the task-specific first stage adapts to both dense congestion fields and sparse DRC maps across technology nodes.

Table 6: Reconstruction quality of the task-specific VAEs

Dataset	Task	MAE ↓	SSIM ↑	Correlation ↑
N28	DRC	0.00095	0.9934	0.9870
	Congestion	0.00132	0.9932	0.9608
N14	DRC	0.00648	0.7009	0.2782
	Congestion	0.00390	0.9676	0.9305

Table 7: Standard metrics on the N28 DRC task.

Method	MAE ↓	NRMS ↓	SSIM ↑	Pearson ↑
RouteNet (SOTA)	0.00313	0.03300	0.95481	0.38156
Pixel Diffusion (9ch)	0.01961	0.20180	0.59270	0.28990
LDM (15ch)	0.00292	0.03373	0.96470	0.50477
LDM+ControlNet (15ch)	0.00280	0.02893	0.96780	0.52483

Table 8: DRC-specific metrics on N28.

Method	TopK@1% ↑	TopK@0.5% ↑	Hotspot-MAE ↓	NZ-Pearson ↑	F1@0.1 ↑
RouteNet	0.24164	0.25454	0.05729	0.39655	0.41343
Pixel Diffusion (9ch)	0.22073	0.24693	0.07683	0.46287	0.15370
LDM (15ch)	0.33837	0.33250	0.06013	0.44733	0.40247
LDM+Cnt (15ch)	0.34940	0.34830	0.05580	0.46143	0.44203

5 Routability Evaluation

We report both standard metrics and task-specific metrics. The standard metrics maintain direct comparability with prior routability literature, while the task-specific metrics reflect how the generated fields are used in physical design. For DRC, the key questions are whether the method localizes violation hotspots and estimates their severity accurately. For congestion, the key questions are whether the generated field captures the spatial distribution of routing demand and remains well aligned in active regions.

5.1 DRC violation map generation on N28

Tables 7 and 8 summarize the DRC results on CircuitNet 2.0 N28 [6], with RouteNet [20] as the deterministic baseline. On the standard metrics, the latent generative models give the strongest overall results, and LDM+ControlNet achieves the best values on all four metrics: MAE = 0.00280, NRMS = 0.02893, SSIM = 0.96780, and Pearson = 0.52483. These values indicate that the generated DRC fields preserve both pixel-level fidelity and global spatial structure.

The task-specific metrics give the clearest view of DRC quality. LDM+ControlNet achieves the best *TopK@1%* and *TopK@0.5%*, with values of 0.34940 and 0.34830, showing precise localization of the highest-risk regions. It also gives the best *Hotspot-MAE*, 0.05580, and the best *F1@0.1*, 0.44203, indicating accurate support and severity modeling in active violation regions. *NZ-Pearson* remains high at 0.46143, which shows that the ranking of nonzero violation regions is also preserved well. Overall, the DRC results highlight the value of task-specific evaluation, since the hotspot-oriented metrics align closely with how DRC maps are used in sign-off analysis.

Table 9: Standard regression metrics on the N28 congestion task. Best results are shown in bold.

Method	MAE ↓	NRMS ↓	SSIM ↑	Pearson ↑
GPDL (SOTA)	0.02958	0.03367	0.90960	0.25433
Pixel Diffusion (3ch)	0.02730	0.03167	0.91990	0.30770
LDM (10ch)	0.02915	0.03380	0.91223	0.33127
LDM+ControlNet (10ch)	0.02859	0.03430	0.90310	0.36870

Table 10: Congestion metrics on N28. For Spatial Bias, values closer to zero are better.

Method	NZ-Pearson ↑	Spatial Bias → 0	Uncertainty ↑
GPDL (SOTA)	0.25487	0.00667	–
Pixel Diffusion (3ch)	0.30883	0.00708	0.24107
LDM (10ch)	0.33167	-0.01259	0.21233
LDM+Cnt (10ch)	0.36923	-0.00441	0.35920

Table 11: N14 DRC generation results.

Method	MAE ↓	SSIM ↑	TopK@1% ↑	Hotspot-MAE ↓	NZ-Pearson ↑	Uncertainty ↑
LDM	0.00627	0.7136	0.0125	0.00571	0.0874	0.4972
LDM+Ctn	0.00633	0.7089	0.0148	0.00578	0.0358	0.4329

5.2 Congestion map generation on N28

Tables 9 and 10 summarize the congestion results on CircuitNet 2.0 N28 [6], with GPDL [9] as the prior baseline. On the standard dense-field metrics, Pixel Diffusion achieves the best MAE, NRMS, and SSIM, with values of 0.02730, 0.03167, and 0.91990, respectively. LDM+ControlNet achieves the best Pearson correlation, 0.36870, indicating the strongest global spatial agreement among the compared methods. The task-specific congestion metrics further clarify the behavior of the generative models. LDM+ControlNet achieves the best NZ-Pearson, 0.36923, the spatial bias closest to zero, -0.00441, and the highest uncertainty score, 0.35920. These values show that its generated congestion fields align well with the active spatial demand pattern, remain close to an unbiased estimate overall, and provide informative uncertainty. Together, the congestion results show that the standard metrics capture dense-field reconstruction quality, while the task-specific metrics capture spatial demand fidelity and uncertainty quality more directly.

5.3 DRC violation map generation on N14

Table 11 shows that the framework transfers to N14 for DRC generation. The unified LDM gives the strongest overall result profile, with the best MAE (0.00627), SSIM (0.7136), Hotspot-MAE (0.00571), NZ-Pearson (0.0874), and uncertainty-error correlation (0.4972). The ControlNet variant gives the best TopK@1% (0.0148), indicating slightly sharper localization of the highest-risk regions. Since the N14 DRC labels are sparse and have a limited dynamic range, the hotspot-oriented metrics are the more informative indicators here. Overall, the N14 results show that the same conditional generation framework remains effective across technology nodes, with the unified latent model giving the more balanced DRC behavior.

Table 12: N14 congestion generation results.

Method	MAE ↓	SSIM ↑	Pearson ↑	Spatial Bias → 0	Uncertainty ↑
LDM	0.03416	0.7678	0.0369	-0.00472	0.0094
LDM+ControlNet	0.03588	0.7654	0.0370	0.00297	0.0197

5.4 Congestion map generation on N14

Table 12 shows stable transfer to N14 for congestion generation. The unified LDM gives the best dense-field reconstruction quality, with the lowest MAE (0.03416) and highest SSIM (0.7678). The ControlNet variant gives the best Pearson correlation (0.0370), the spatial bias closest to zero (0.00297), and the highest uncertainty-error correlation (0.0197). These numbers separate the roles of the two variants clearly: the unified model gives slightly better reconstruction fidelity, while the ControlNet model gives slightly better global alignment and uncertainty behavior. Taken together, the N14 results show that the same conditional generative framework remains usable across both N28 and N14.

Across both tasks, the N28 evaluation highlights two complementary aspects of routability-map generation. The first is standard fidelity, which enables direct comparison under a shared metric set. The second is task-aligned utility, which measures whether the generated maps emphasize the spatial regions that matter most for routing analysis. This distinction is especially important for DRC, where sparse violation structure makes hotspot-focused evaluation essential, and it remains equally informative for congestion, where the accuracy of high-demand regions and systematic demand balance strongly influence practical usefulness. The two-table evaluation protocol therefore provides a consistent and physically meaningful view of generation quality across both tasks.

6 Conclusion

This paper presents *CLDRoute*, a conditional latent diffusion framework for routability map generation during physical design. Instead of treating routability estimation as deterministic prediction, CLDRoute models both congestion and DRC as spatially structured routability fields and generates them conditionally from placement-stage routing controls. The method combines physics-aware routing controls, task-specific latent encoding, and multi-scale conditional diffusion to handle the different structures of dense congestion maps and sparse DRC violation maps within one unified framework. Across the experiments, the results show that this formulation supports accurate generation on CircuitNet 2.0 N28 and also transfers to N14, while providing uncertainty estimates through sample-based inference. Overall, CLDRoute provides a practical placement-stage view of routability by generating both an expected routability map and a spatial uncertainty map from the same design representation.

References

- [1] Kyeonghyeon Baek, Hyunbum Park, Suwan Kim, Kyumyung Choi, and Taewhan Kim. 2022. Pin accessibility and routing congestion aware DRC hotspot prediction using graph neural network and U-Net. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [2] Zhuomin Chai, Yuxiang Zhao, Wei Liu, Yibo Lin, Runsheng Wang, and Ru Huang. 2023. Circuitnet: An open-source dataset for machine learning in vlsi cad applications with improved domain-specific evaluation metric and learning

- strategies. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 5034–5047.
- [3] Amur Ghose, Vincent Zhang, Yingxue Zhang, Dong Li, Wulong Liu, and Mark Coates. 2021. Generalizable cross-graph embedding for gnn-based congestion prediction. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
 - [4] Jonathan Ho, Ajay Jain, and P. Abbeel. 2020. Denoising Diffusion Probabilistic Models. *ArXiv abs/2006.11239* (2020). <https://api.semanticscholar.org/CorpusID:219955663>
 - [5] Yuma Ichikawa and Koji Hukushima. 2024. Learning dynamics in linear vae: Posterior collapse threshold, superfluous latent space pitfalls, and speedup with kl annealing. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1936–1944.
 - [6] Xun Jiang, Zhuomin Chai, Yuxiang Zhao, Yibo Lin, Runsheng Wang, Ru Huang, et al. 2024. Circuitnet 2.0: An advanced dataset for promoting machine learning innovations in realistic chip design environment. In *The Twelfth International Conference on Learning Representations*.
 - [7] Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).
 - [8] Rongjian Liang, Hua Xiang, Jinwook Jung, Jiang Hu, and Gi-Joon Nam. 2022. A stochastic approach to handle non-determinism in deep learning-based design rule violation predictions. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–8.
 - [9] Siting Liu, Qi Sun, Peiyu Liao, Yibo Lin, and Bei Yu. 2021. Global Placement with Deep Learning-Enabled Explicit Routability Optimization. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1821–1824. doi:10.23919/DATeS1398.2021.9473959
 - [10] Wen-Hao Liu, Wei-Chun Kao, Yih-Lang Li, and Kai-Yuan Chao. 2013. NCTU-GR 2.0: Multithreaded collision-aware global routing with bounded-length maze routing. *IEEE Transactions on computer-aided design of integrated circuits and systems* 32, 5 (2013), 709–722.
 - [11] James Lucas, George Tucker, Roger B Grosse, and Mohammad Norouzi. 2019. Don't blame the elbo! a linear vae perspective on posterior collapse. *Advances in Neural Information Processing Systems* 32 (2019).
 - [12] Nicole Meng, Caleb Manicke, Ronak Sahu, Caiwen Ding, and Yingjie Lao. 2025. Advancing Adversarial Robustness in GNeRFs: The IL2-NeRF Attack. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 16388–16397.
 - [13] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, Will Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A graph placement methodology for fast chip design. *Nature* 594 (2021), 207 – 212. <https://api.semanticscholar.org/CorpusID:235395490>
 - [14] Min Pan and Chris Chu. 2006. FastRoute: A step to integrate global routing into placement. In *Proceedings of the 2006 IEEE/ACM international conference on Computer-aided design*. 464–471.
 - [15] Robin Rombach, A. Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2021. High-Resolution Image Synthesis with Latent Diffusion Models. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 10674–10685. <https://api.semanticscholar.org/CorpusID:245335280>
 - [16] Tim Salimans and Jonathan Ho. 2022. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512* (2022).
 - [17] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502* (2020).
 - [18] Peter Spindler and Frank M Johannes. 2007. Fast and accurate routing demand estimation for efficient routability-driven placement. In *2007 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 1–6.
 - [19] Zhiyao Xie, Yu-Hung Huang, Guan-Qi Fang, Haoxing Ren, Shao-Yun Fang, Yiran Chen, and Nvidia Corporation. 2018. RouteNet: Routability prediction for mixed-size designs using convolutional neural network. In *Proceedings of the International Conference on Computer-Aided Design*. 1–8.
 - [20] Zhiyao Xie, Yu-Hung Huang, Guan-Qi Fang, Haoxing Ren, Shao-Yun Fang, Yiran Chen, and Jiang Hu. 2018. RouteNet: Routability prediction for Mixed-Size Designs Using Convolutional Neural Network. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. doi:10.1145/3240765.3240843
 - [21] Shuwen Yang, Zhihao Yang, Dong Li, Yingxueff Zhang, Zhanguang Zhang, Guojie Song, and Jianye Hao. 2022. Versatile multi-stage graph neural network for circuit representation. *Advances in Neural Information Processing Systems* 35 (2022), 20313–20324.
 - [22] Cunxi Yu and Zhiru Zhang. 2019. Painting on placement: Forecasting routing congestion using conditional generative adversarial nets. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
 - [23] Yuxiang Zhao, Xun Jiang, Qiang Xu, Runsheng Wang, Yibo Lin, et al. 2025. DeepLayout: Learning Neural Representations of Circuit Placement Layout. In *Forty-second International Conference on Machine Learning*.
 - [24] Lancheng Zou, Su Zheng, Peng Xu, Siting Liu, Bei Yu, and Martin DF Wong. 2025. Lay-Net: Grafting Netlist Knowledge on Layout-Based Congestion Prediction. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*